

پاسخ‌ها لزوماً یکتا نیستند و پاسخ‌های درست دیگر نیز در نظر گرفته می‌شوند.

(۱)

الف) سرعت کاهش دما را زیاد می‌کنیم تا حرکات بد کمتر شود و سریع‌تر به همگرایی برسیم.

ب) در این حالت الگوریتم تپه‌نوردی بهتر عمل می‌کند زیرا هیچ حرکت بدی انجام نمی‌دهد و با بیشترین شیب به سمت اکسترمم سراسری می‌رود.

ج) دما را افزایش می‌دهیم و سرعت کاهش دما را کم می‌کنیم تا احتمال انجام حرکات بد بیشتر شود تا با انجام این حرکات از کمینه‌های محلی فرار کنیم.

(۲)

(الف) به صورت یک رشته بیت n تایی از صفر و یک‌ها به طوری که بیت i ام نشان‌دهنده خرید (1) یا عدم خرید (0) بسته i ام است.

(ب)

$$F(C_1 C_2 \dots C_n) = \begin{cases} 0 & \text{اگر همه ابزارها تهیه نشده باشند} \\ \frac{1}{\sum_{i=1}^n C_i} & \text{اگر همه ابزارها تهیه شده باشند} \end{cases}$$

دو حالت در نظر می‌گیریم.

حالت اول) اگر با توجه به رشته صفر و یک‌ها (که نشان می‌دهد کدام بسته‌ها خریداری شده) همه ابزارها تهیه نشده باشند، مقدار تابع شایستگی برابر صفر خواهد بود.

حالت دوم) اگر با توجه به رشته صفر و یک‌ها (که نشان می‌دهد کدام بسته‌ها خریداری شده) همه ابزارها تهیه شده باشند، مقدار تابع شایستگی برابر با معکوس تعداد بسته‌های خریداری شده خواهد بود.

(ج) عملگر ترکیب از روش ترکیب تک‌نقطه استفاده می‌کنیم، به این صورت که یک عدد به صورت تصادفی i بین 1 تا n انتخاب می‌کنیم و سپس مطابق زیر از دو کروموزوم اولیه دو فرزند جدید می‌سازیم:

$$\left. \begin{array}{l} C_{11} C_{12} \dots C_{1n} \\ C_{21} C_{22} \dots C_{2n} \end{array} \right\} \begin{array}{l} C_{11} C_{12} \dots C_{1i} C_{2i+1} \dots C_{2n} \\ C_{21} C_{22} \dots C_{2i} C_{1i+1} \dots C_{1n} \end{array}$$

عملگر جهش) هر یک از کروموزوم‌ها با یک احتمال کوچک ممکن است جهش یابند. برای جهش یک کروموزوم کافی است ابتدا به صورت تصادفی یکی از بیت‌های آن را انتخاب کنیم سپس مقدار آن بیت را تغییر دهیم. (اگر صفر بود به یک و اگر یک بود به صفر تغییر می‌کند).

(۳)

الف) با انجام الگوریتم در مرحله ی ۱۶م به هدف می‌رسیم.

مرحله	خانه اول	خانه دوم
0	A	E
1	C	D
2	B	F
3	N	G
4	F	J
5	G	L
6	F	M

ب) با انجام الگوریتم به این نتیجه می‌رسیم که الگوریتم در اکستریم محلی گیر می‌کند و مدام از خانه‌های F,O به G,P و سپس دوباره به F,O می‌رود و به خانه هدف (M) نمی‌رسیم.

مرحله	خانه اول	خانه دوم
0	A	E
1	C	D
2	B	F
3	N	G
4	F	O
5	G	P
6	F	O

الف) تعداد وضعیت‌های منحصر به فرد مجاور

در هر وضعیت به طول n ، عمل مجاز جابجایی دو عنصر «همسایه» است. جفت‌های همسایه برای جابجایی عبارتند از:

$$(i_1, i_2), (i_2, i_3), \dots, (i_{n-1}, i_n)$$

بنابراین تعداد کل همسایه‌های ممکن (عمل‌های مجاز) برابر با $n - 1$ است.

در این مسئله چون $n = 5$ است، در هر مرحله ۴ وضعیت مجاور (همسایه) داریم.

ب) تعریف تابع هدف (Objective Function)

تابع هدف را تعداد زوج‌های نامرتب (Inversions) در نظر می‌گیریم. یعنی مجموع جفت‌هایی که در آن، عدد بزرگتر در سمت چپ عدد کوچکتر قرار گرفته است. هدف مسئله، کمینه کردن (Minimize) این تابع است.

محاسبه برای وضعیت اولیه $[5, 1, 4, 2, 3]$:

- بررسی عدد ۵: جفت‌های $(5, 1), (5, 4), (5, 2), (5, 3)$ نامرتب هستند (۴ مورد)
- بررسی عدد ۱: جفت نامرتب ندارد.
- بررسی عدد ۴: جفت‌های $(4, 2), (4, 3)$ نامرتب هستند (۲ مورد)
- بررسی عدد ۲: جفت نامرتب ندارد.
- بررسی عدد ۳: جفت نامرتب ندارد.

مقدار کل تابع هدف در وضعیت شروع: $f = 4 + 2 = 6$

ج) بررسی وجود مینیمم محلی

خیر، این تابع هدف مینیمم محلی ندارد. استدلال: در هر وضعیتی که دنباله کاملاً مرتب نباشد، قطعاً حداقل یک جفت همسایه مانند (a_i, a_{i+1}) وجود دارد که در آن $a_i > a_{i+1}$ است. با جابجا کردن این دو عنصر، تعداد زوج‌های نامرتب (Inversions) دقیقاً ۱ واحد کاهش می‌یابد. از آنجا که در هر وضعیت نامرتب، همیشه راهی برای بهبود تابع هدف وجود دارد، الگوریتم در هیچ نقطه‌ای به جز وضعیت هدف (آرایه مرتب با مقدار صفر) متوقف نمی‌شود.

د) اجرای دو مرحله از الگوریتم‌ها**۱. الگوریتم تپه‌نوردی (Hill Climbing)**

این الگوریتم در هر مرحله همسایه‌ای را انتخاب می‌کند که بیشترین بهبود (کمترین مقدار تابع هدف) را داشته باشد.

- **وضعیت شروع:** $[5, 1, 4, 2, 3]$ با مقدار $f = 6$
- **مرحله اول:** بررسی همسایه‌ها:

- جابجایی ۵ و ۱: $[1, 5, 4, 2, 3] \rightarrow f = 5$ (انتخاب می‌شود)
- جابجایی ۱ و ۴: $[5, 4, 1, 2, 3] \rightarrow f = 7$
- جابجایی ۴ و ۲: $[5, 1, 2, 4, 3] \rightarrow f = 5$
- جابجایی ۲ و ۳: $[5, 1, 4, 3, 2] \rightarrow f = 7$
- **نتیجه:** وضعیت به $[1, 5, 4, 2, 3]$ تغییر می‌کند با $f = 5$.

- **مرحله دوم:** بررسی همسایه‌های وضعیت جدید:

- جابجایی ۵ و ۴: $[1, 4, 5, 2, 3] \rightarrow f = 4$

- نتیجه نهایی: وضعیت به $[1, 4, 5, 2, 3]$ می‌رسیم.

۲. الگوریتم سرد کردن تدریجی (Simulated Annealing)

در این الگوریتم انتخاب‌ها تصادفی است و بر اساس پارامتر دما (T) احتمال پذیرش وضعیت بدتر وجود دارد. فرض کنیم $T = 10$ و ضریب کاهش دما 0.9 باشد.

- مرحله اول:

* وضعیت فعلی: $[5, 1, 4, 2, 3]$ با $f = 6$

* انتخاب تصادفی همسایه (مثلاً جابجایی ۱ و ۴): $[5, 4, 1, 2, 3]$ با $f = 7$.

* چون وضعیت بدتر شد، احتمال پذیرش را محاسبه می‌کنیم: $P = e^{\Delta E/T} = e^{(6-7)/10} = e^{-0.1} \approx 0.90$

* با فرض تولید عدد تصادفی 0.45 (که کوچکتر از 0.90 است)، این وضعیت بدتر را می‌پذیریم.

* وضعیت جدید: $[5, 4, 1, 2, 3]$ با $f = 7$.

- مرحله دوم:

* به‌روزرسانی دما: $T = 10 \times 0.9 = 9$

* انتخاب تصادفی همسایه (مثلاً جابجایی ۵ و ۴): $[4, 5, 1, 2, 3]$ با $f = 6$.

* چون مقدار تابع هدف بهبود یافت ($6 < 7$)، وضعیت جدید را بدون قید و شرط می‌پذیریم.

* وضعیت جدید: $[4, 5, 1, 2, 3]$ با مقدار $f = 6$.

تحلیل مسئله

- حالت آغازین (چپ): چیدمان اعداد به صورت نامنظم است.
- حالت نهایی یا هدف (راست): اعداد ۱ تا ۸ به ترتیب چیده شده‌اند و خانه خالی در گوشه پایین سمت راست قرار دارد.
- تابع هزینه (Cost/Heuristic): مجموع فواصل منتهن (Manhattan Distance)؛ این تابع یعنی برای هر عدد، تعداد خانه‌هایی که باید طی کند تا به جایگاه اصلی‌اش در جدول هدف برسد را حساب کرده و با هم جمع می‌کنیم.

پاسخ بخش (الف): روش Hill Climbing

۱. آیا لزوماً جواب بهینه به دست می‌آید؟
خیر. الگوریتم تپه‌نوردی یا Hill Climbing یک الگوریتم جستجوی محلی است که همواره به سمتی می‌رود که مقدار تابع هزینه کاهش یابد (بهترین همسایه فعلی). این روش با مشکلات زیر روبروست که مانع رسیدن به بهینه مطلق (Global Optimum) می‌شود:
 - اکسترمم محلی (Local Maxima/Minima): وضعیتی که از تمام همسایه‌ها بهتر است اما هدف نیست.
 - شانه (Plateau): منطقه‌ای که مقدار تابع هزینه برای همسایه‌ها برابر است و الگوریتم "گیج" می‌شود.
 - ستیخ (Ridge): مسیری که به سمت قله می‌رود اما شیب آن به گونه‌ای است که حرکت مستقیم به سمت بالا سخت است.
۲. جواب غیر بهینه چطور به دست می‌آید؟
اگر الگوریتم در نقطه‌ای قرار بگیرد که هیچ‌کدام از حرکت‌های مجاز (بالا، پایین، چپ، راست کردن خانه خالی) منجر به کاهش مجموع فواصل منتهن نشود، الگوریتم در همان نقطه متوقف می‌شود. در این حالت، ما به یک اکسترمم محلی رسیده‌ایم. در واقع الگوریتم به جای رسیدن به هدف (که هزینه آن صفر است)، در حالتی با هزینه بیشتر (مثلاً ۴ یا ۵) متوقف می‌شود چون هر حرکتی از آن نقطه، هزینه را بدتر (بیشتر) می‌کند.

پاسخ بخش (ب): معادل بودن با جستجوی اول بهترین حریصانه

- آیا این دو روش معادل هستند؟
بله، از نظر استفاده از تابع هیوریستیک $h(n)$ (در اینجا مجموع فواصل منتهن) معادل هستند اما در استراتژی جستجو تفاوت دارند.
- شباهت: هر دو روش از تابع هیوریستیک $h(n)$ (در اینجا مجموع فواصل منتهن) برای انتخاب حرکت بعدی استفاده می‌کنند. هر دو "حریصانه" عمل می‌کنند یعنی در هر گام وضعیتی را انتخاب می‌کنند که به نظر می‌رسد به هدف نزدیک‌تر است.
 - تفاوت اصلی:
 - Hill Climbing: حافظه ندارد. فقط وضعیت فعلی را می‌بیند و به بهترین همسایه می‌رود. اگر به بن‌بست بخورد، نمی‌تواند به عقب برگردد (Backtrack).

– Greedy Best-First Search: تمام وضعیت‌های مشاهده شده را در یک لیست (Frontier) نگه می‌دارد. اگر مسیری به بن‌بست بخورد، می‌تواند بهترین وضعیت بعدی موجود در کل حافظه را انتخاب کند.

• نتیجه‌گیری: اگر منظور سوال این است که آیا هر دو از یک منطق برای ارزیابی وضعیت استفاده می‌کنند، پاسخ مثبت است. اما در عمل، Greedy Search به دلیل داشتن حافظه، شانس بیشتری برای خروج از بن‌بست‌ها نسبت به Hill Climbing ساده دارد.

محاسبه هیوریستیک حالت آغازین (برای درک بهتر):

• عدد ۱: فاصله تا جای اصلی $(0, 0) = ۲$ واحد

• عدد ۲: فاصله تا جای اصلی $(0, 1) = ۱$ واحد

• ... به همین ترتیب مجموع کل محاسبه می‌شود.

الگوریتم سعی می‌کند در هر مرحله این عدد کل را به صفر برساند.